

PIPELYT

Technical Project Overview

An agentic AI platform for social media — how it works, what the agents do, how it is built, and how the code is organized.

FastAPI • Google Gemini 2.5 Flash • React • PostgreSQL • AWS Lambda

OVERVIEW

What Pipelyt Is

Pipelyt takes a single-sentence campaign brief and turns it into platform-tailored content and AI-generated visuals — then publishes, schedules, monitors and replies to engagement automatically.

The defining idea: the system is agentic. The work is not done by one large prompt, but by a pipeline of specialized AI agents — each with one narrow job — where the output of one agent becomes the input of the next.

1

SENTENCE BRIEF IN

4

PLATFORMS OUT — LINKEDIN • X • FACEBOOK •
INSTAGRAM

THE PROBLEM

Why Not Just One AI Prompt?

A single prompt that tries to do everything at once fails in predictable ways:

Invents facts

Hallucinated numbers and claims that were never true.

Drifts off-brand

Wrong tone, wrong voice, wrong colours.

Generic copy

Reads identically on every channel — nothing native.

THE PIPELYT ANSWER

Decompose the work the way a real content team is structured — strategist, researcher, copywriter, designer, editor — and give each role to a dedicated agent. A narrow job is far easier for a model to do reliably, and every step can be inspected, tested and improved on its own.

System Architecture at a Glance

One monorepo, three applications. The agentic engine lives entirely in the backend; the frontends consume it over a normal HTTP API.

Backend

apps/backend

FastAPI + Python. Hosts every agent, the database layer and all integrations. Runs on AWS Lambda in production.

Product dashboard

apps/product-page

React 18. Where users write briefs, review generated content, schedule posts and read analytics.

Landing page

apps/landing-page

React 19. The marketing site and sign-up / authentication screens.

Rule of the codebase: routers stay thin and only handle HTTP — every integration and all real logic sits behind a service module.

THE CORE

The Multi-Agent Engine

Three things to understand, in order: how a single agent is built, how the agents are wired together, and what each one actually does.

How a Single Agent Is Built

agent = role + structured input/output contract + one model call

1 Python function

Assembles a carefully written prompt — a role definition plus a strict input / output contract.

2 `_call_agent()`

One shared helper sends that prompt to the model and parses the reply.

3 Gemini 2.5 Flash

The model returns a JSON response for that single, narrow task.

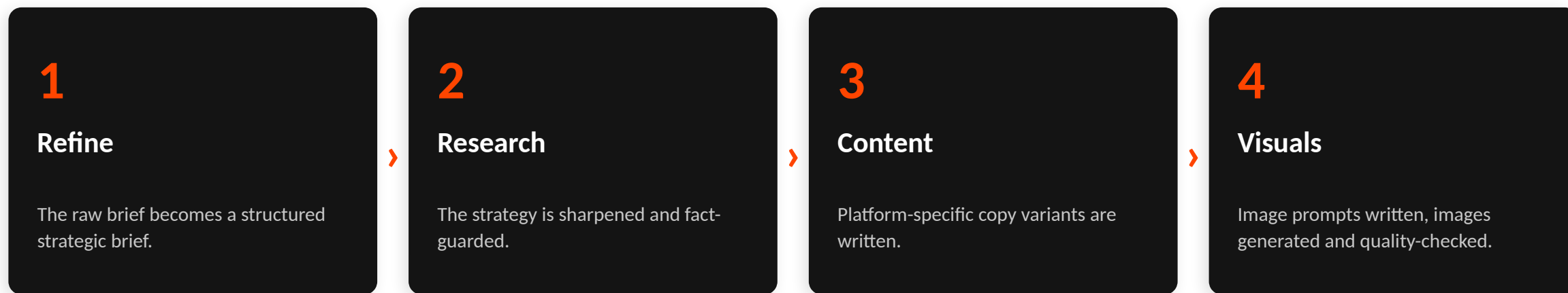
4 Parsed dict out

A clean Python dictionary — the exact input for the next agent.

Agents share no global state. Each receives exactly the structured output of the stage before it — which makes the pipeline easy to debug and lets every agent be tested on its own.

How the Agents Are Wired Together

A single orchestrator runs the agents in a fixed sequence, passing each result forward:



Every stage is individually timed

so the team can see exactly where generation time is spent.

Context-aware

Text, video and document posts skip the visual stages — saving ~1 minute of image generation.

Seven Agents — the Makers

1

Brief Refiner

STRATEGIST

Rewrites the raw one-line brief into a structured strategic brief — audience, goal, tone and angle made explicit, so no later agent has to guess.

2

Research Analyst

FACT GUARD

Sharpens the strategy into three testable angles and builds a do-not-claim list of facts the writer must never invent. The anti-hallucination guardrail.

3

Content Copywriter

WRITER

Classifies the company, then writes three copy variants per platform — long-form for LinkedIn, punchy for X, conversational for Facebook, hashtag-led for Instagram.

4

Visualist

DESIGNER

Designs the post image — writes the AI image-generation prompt and the on-image overlay copy, anchored to the brand's real colours and logo.

Agents 1 – 4 build the campaign.

Seven Agents — the Checkers & Planner



Visual Critic

QUALITY GATE

Audits every raw AI-generated image for quality before any overlay. A failed audit triggers automatic regeneration.



Strategic Critic

EDITOR

Reviews the finished payload — copy and visuals together — for growth fitness. A final editorial pass.



Campaign Planner

SCHEDULER

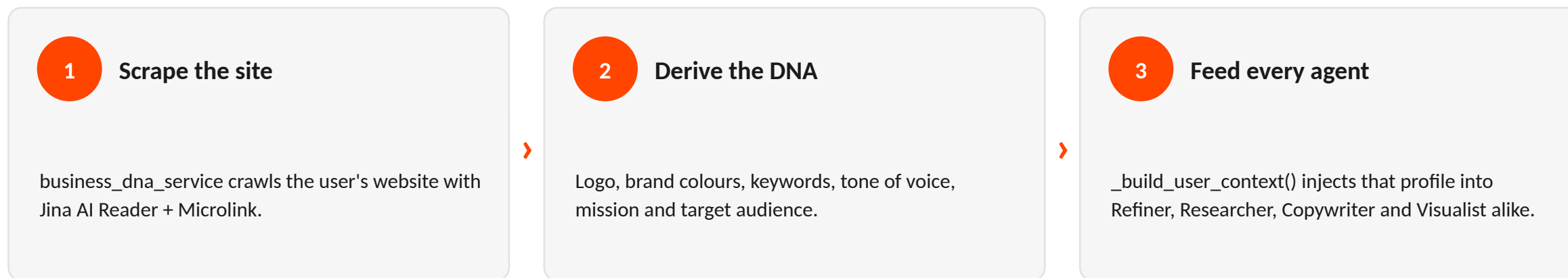
Expands one brief into a multi-day, multi-platform content calendar — a whole week of posts from a single idea.

SELF-CORRECTING BY DESIGN

The Visual Critic forms a feedback loop with the image generator — a poor image is regenerated, not shipped. Quality is enforced by an agent, not by a person reviewing every output. This is what lets Pipelyt run with little human supervision.

Business DNA — Shared Brand Context

On its own, a pipeline of agents would still produce generic work. Business DNA is the context layer that keeps every output on-brand.



This is why generated posts sound like the brand and generated images use the brand's real colours and logo — without the user specifying any of it each time.

THE FULL PLATFORM

Beyond Generation

The agents create content; the surrounding services make it a complete product.

Publishing & scheduling

OAuth to every network; posts text, image, video and PDF carousels. An EventBridge heartbeat auto-publishes due posts every minute, in the user's timezone.

Analytics

Periodic metric syncs feed dashboards for followers, engagement, reach and per-platform performance.

Reputation intelligence

Reads comments, scores sentiment with a Semantic Score, and drafts on-brand replies — sent in bulk or one at a time.

Team workspace

Multi-tenant, with a separate Business DNA per brand, role-based permissions and per-member content scoping for agencies.

How the Project Is Organized

apps/backend

FastAPI service — hosts the entire agentic engine. Deploys to AWS Lambda.

apps/product-page

The authenticated React 18 dashboard.

apps/landing-page

Marketing site and authentication (React 19).

backend/services/

One module per concern — ai_service (the agents), business_dna_service, social_service.

backend/routers/

Thin HTTP endpoints grouped by area — auth, content, analytics, payment, profile.

backend/models.py

The database schema — users, accounts, posts, drafts, scheduled posts, analytics.

Conventions: new integrations go behind a service module — routers never call third-party APIs directly; schema changes ship with a self-repairing migration.

How It Is Deployed

Backend

Packaged as a container image on AWS Lambda via the Mangum adapter.

Scheduler

A second Lambda, triggered by EventBridge every minute, fires due scheduled posts.

Frontends

Both React apps build with Vite and deploy through AWS Amplify.

External services

Gemini for content & images, S3 for storage, Stripe for payments, the networks' OAuth APIs.

CONCLUSION

Why the Agentic Design Pays Off

1

Reliability

The do-not-claim list and two Critic agents catch invented facts and weak output before a user sees them.

2

Consistency

Business DNA is injected into every agent — staying on-brand is the default, not a correction.

3

Maintainability

Each agent is a small, single-purpose function with a clear contract — improve one without touching the rest.

4

Performance

Per-stage timing and stage-skipping for non-image posts keep generation fast where it can be.

5

Extensibility

New roles can be added as new agents without rewriting what already works.

Pipelyt turns one sentence into a finished, on-brand, multi-platform campaign by running it through a team of AI specialists — each doing one job well, each checking the work that came before.